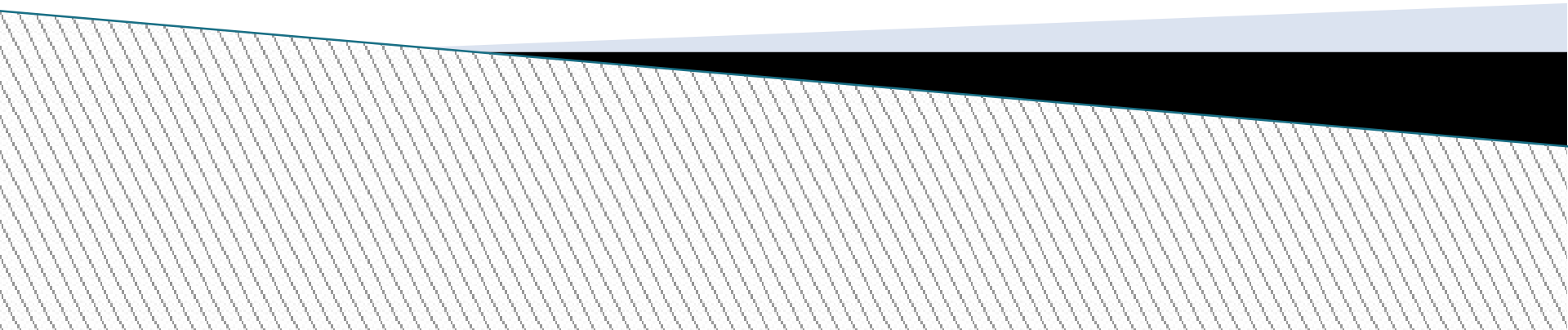


# Arquitectura de Computadoras

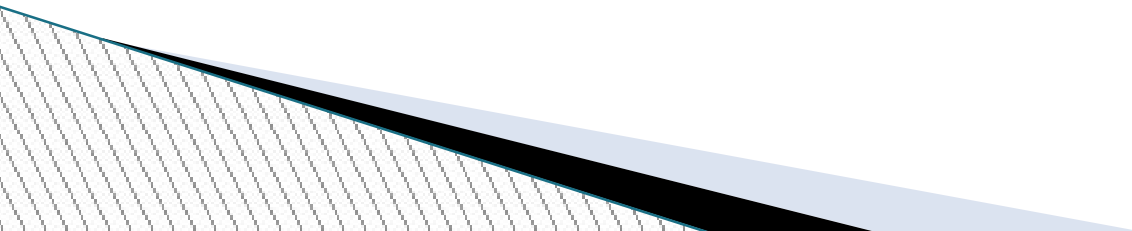
## Taller de MIPS

(cap. 2 del libro estructura y diseño de  
computadores de Patterson y Hennessy )

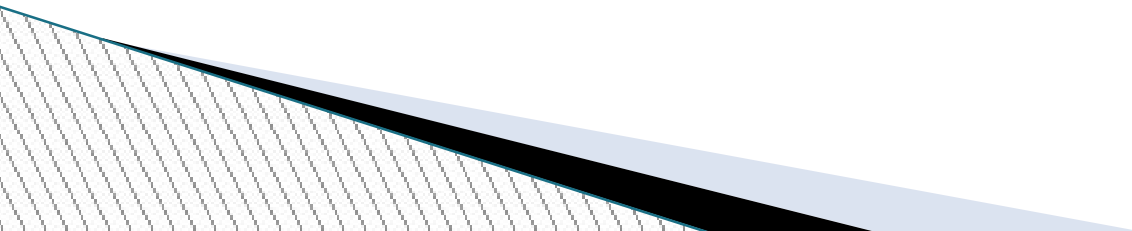
2020 – silvina meinero



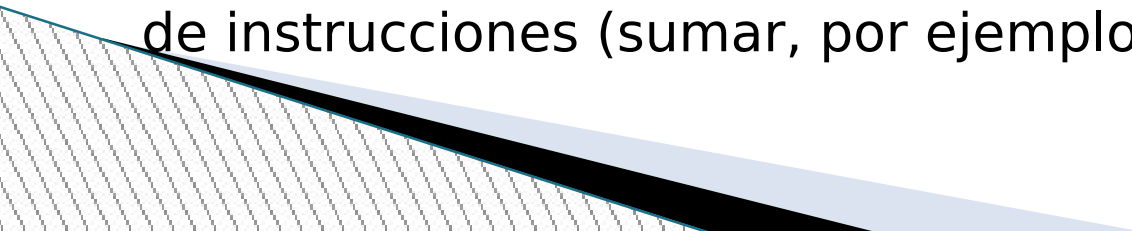
# Lenguajes para la máquina

- 1 Las computadoras son circuitos electrónicos, por lo tanto para hablar con ellas se necesita enviarles señales eléctricas.
  - ▯ Las señales viajan por los circuitos electrónico que están formados por (millones de) transistores.
  - ▯ Los transistores son interruptores que, en función del valor de la señal eléctrica que los controla, dejan o no pasar esa señal. Por eso a su salida habrá dos posibles valores: estos valores se pueden representar simbólicamente con los dígitos cero y uno.
  - ▯ Los símbolos cero y uno se llaman bits.
- 

# Transistores: ¿que podemos hacer con algunos pocos de ellos?

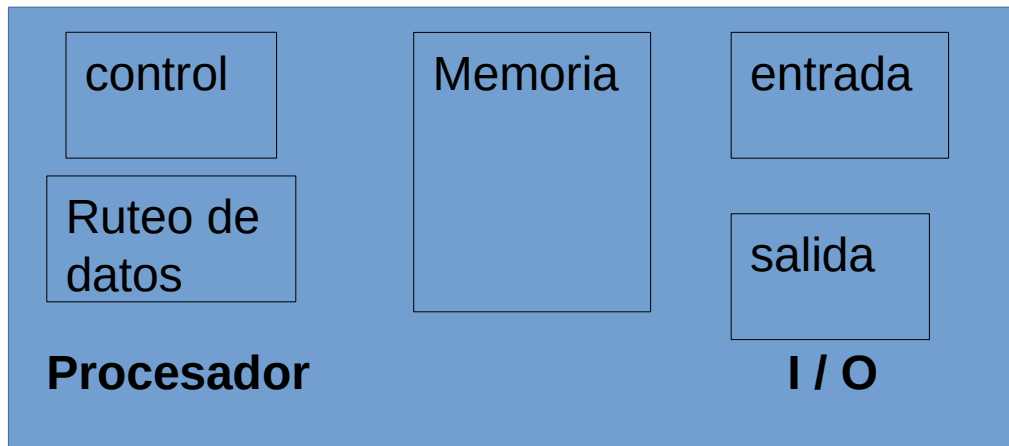
- ▯ Con sólo dos: convertir un 1 en un 0 (y viceversa): hacer una puerta NOT.
  - ▯ Con 4 transistores podemos hacer una puerta lógica NAND o NOR.
  - ▯ Con estas puertas elementales se puede construir cualquier función.
  - ▯ Un computador no es más que un conjunto (grande) de funciones de un conjunto (grande) de entradas (señales).
- 

# Lenguajes para la máquina

- En realidad, el lenguaje máquina está formado por números en base 2. Tanto los datos como las instrucciones se codifican como números en base 2.
  - Cada máquina (o familia de máquinas) tiene su propio lenguaje máquina. Los lenguajes máquina, en realidad, no difieren mucho entre sí, debido a que:
    - – la tecnología de implementación de los circuitos es la misma para todos los fabricantes.
    - – Todas las máquinas deben proporcionar un repertorio básico de instrucciones (sumar, por ejemplo).
- 

# Lenguajes Máquina

- Una computadora, está formada por



- La Unidad de Proceso y Control (es donde nos vamos a centrar en este curso) implementa un conjunto de funciones (de propósito general) a través de millones de transistores.
- Imposible de entender mirando lo que hace cada transistor necesitamos una abstracción. →

# Abstracciones

High-level  
language  
program  
(in C)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compilador C

C compiler  
Assembly  
language  
program  
(for MIPS)

```
swap:
    muli $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    Jr $31
```

Assembler

Assembler  
Binary machine  
language  
program  
(for MIPS)

```
0000000001010000100000000000011000
000000000100011100001100000100001
1000110001100010000000000000000000
1000110011110010000000000000000100
1010110011110010000000000000000000
1010110001100010000000000000000100
0000000111110000000000000000000100
```

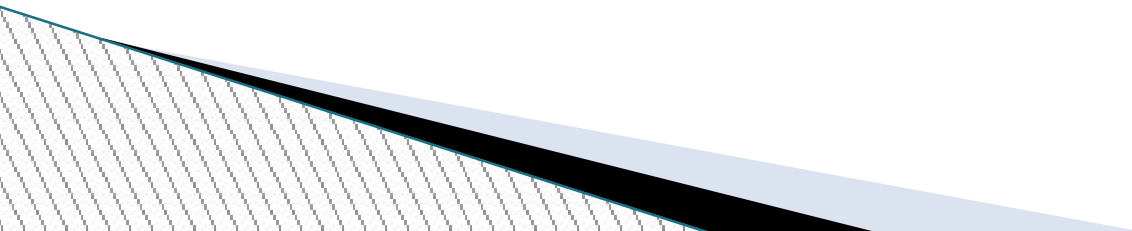
- Lenguajes de alto nivel permiten:

- Facilidad al programar
- Menor tiempo de desarrollo (mayor productividad)
- Independencia del computador

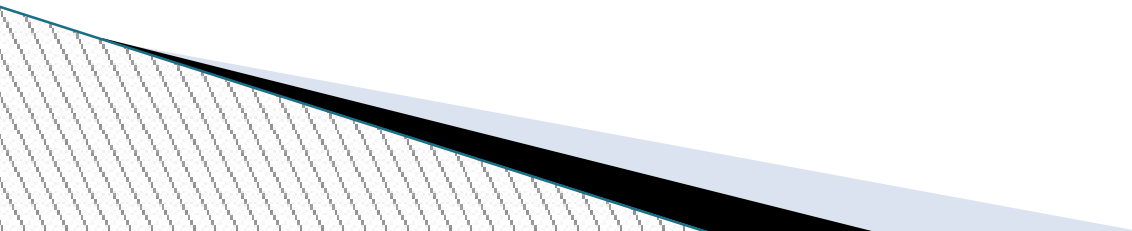
Lenguaje ensamblador:  
– Notación más humana que los números binarios

- Requiere una instrucción por línea

# MIPS

- ▶ MIPS (**M**icroprocessor without **I**nterlocked **P**ipeline **S**tages) familia de microprocesadores de arquitectura RISC.
  - ▶ Diseñado por John Hennessy y su equipo de trabajo en la Universidad de Stanford en el año 1981.
  - ▶ Idea básica del diseño mejorar notoriamente el rendimiento acompañado con reducción de costos.
  - ▶ Puesta en el mercado el año 1985 por MIPS Computer Systems, Inc.(actualmente MIPS Technologies, Inc.).
- 

# MIPS

- ▶ Procesadores MIPS usados por: SGI(Silicon Graphics), DEC, Sony, NEC, Nintendo, TCL Corporation.
  - ▶ Procesadores MIPS usados en:
    - ▢ Reproductores de discos Blu-Ray
    - ▢ Teléfonos inteligentes (TCL)
    - ▢ Nintendo, PlayStation Sony
    - ▢ Decodificadores Motorola y Cisco
    - ▢ Televisores Digitales Samsung y LG
- 



# MIPS

- ▶ Procesadores MIPS usados en: (cont.)
  - ▮ Routers Cisco, NetGear y Linksys
  - ▮ Cámaras digitales Nikon, Sony entre otros.
  - ▮ Equipos de infraestructura de redes (Cavium Redes y Netlogic Microsystems)
  - ▮ Automóviles de Toyota, Volvo, Cadillac entre otros.
  - ▮ Controladoras de impresoras láser

# En línea de tiempo

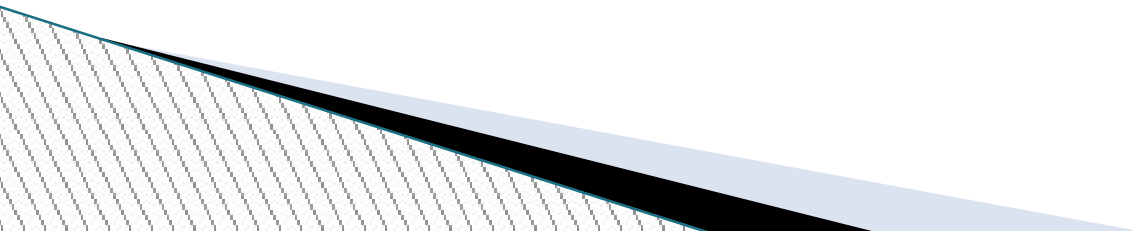
**1981:** Dr. John Hennessy en la Universidad de Stanford lleva la investigación en la construcción de un microprocesador RISC.

**1984:** MIPS Computer Systems, Inc. co-fundada por el Dr. John Hennessy, Skip Stritter (Motorola), y el Dr. John Moussouris(IBM)

**1985:** R2000 microprocesador

**1988:** R3000 microprocesador (usados en equipos SGI y DEC Digital Equipment Corporation).

**1991:** R4000 microprocesador

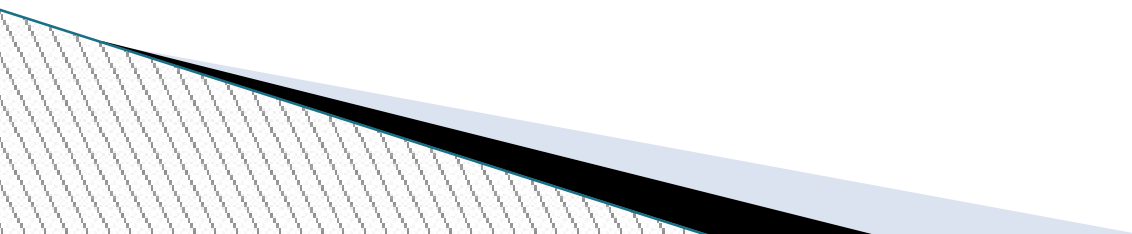


# En línea de tiempo (cont.)

1994: R8000 diseño superescalar

1996: R10000, y sus versiones R12000, R14000, R16000, R16000A (comienza a licenciar sus arquitecturas)

- Las arquitecturas fueron evolucionado: MIPSII, MIPSIII, MIPSIV, MIPSV
- MIPS32 (basado en MIPSII) y MIPS64 (basado en MIPSV)



# Operaciones Aritmética

- Principio de diseño 1: La simplicidad favorece la regularidad.
- La regularidad hace que la implementación sea más simple.
- La simplicidad permite un mayor rendimiento a menor costo.
- Sumar y restar 3 operandos. Dos fuentes y un destino:

ejemplo

código C:  $a = b + c$ ;

código MIPS:

**add \$s0, \$s1, \$s2 # en \$s0 obtenemos \$s1+\$s2**

- Todas las operaciones aritmética tienen esta forma.

# Operaciones aritméticas (cont.)

- Las instrucciones aritméticas utilizan operadores asociados a registros.
- MIPS tiene 32 registros de 32 bits c/u:
- datos de acceso frecuente
- numerados de 0..31
- cada registro es llamado palabra
- Nombre de Registros:
- \$t0, ....., \$t9 temporal
- \$s0, ....., \$s7 temporal salvado
- Principio de diseño 2: cuanto más pequeño más rápido.

# Operaciones aritméticas: ejemplo

► Código C:

```
f = (g + h) - (i + j);
```

Código MIPS:

```
add $t0, $s1, $s2
```

```
add $t1, $s3, $s4
```

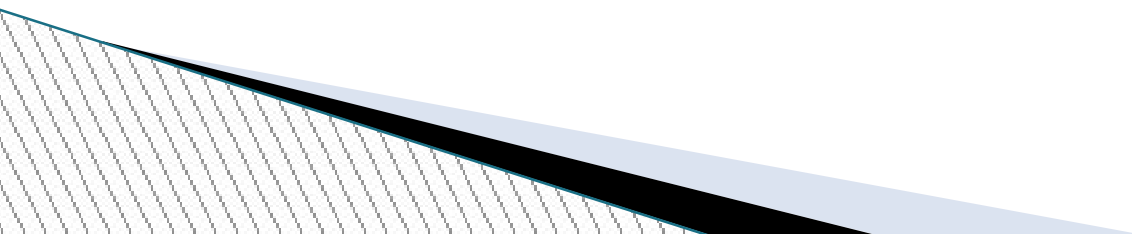
```
sub $s0, $t0, $t1
```

# Operadores de Memoria

- La memoria principal es usada para datos compuestos: arreglos, estructuras, datos dinámicos.
- Para realizar operaciones aritméticas los valores deben ser cargados de la memoria a registros. Y el resultado ser almacenado en memoria.
- La memoria se puede ver como un vector de:
  - $2^{32}$  bytes con direcciones (de byte) que van desde 0 a  $2^{32}-1$
  - $2^{30}$  words con direcciones (de byte) 0, 4, 8, ...  $2^{32}-4$

# Operadores de Memoria (cont.)

- MIPS “big endian” (en contraposición con “little endian”): consiste en almacenar el byte de más peso del word en la dirección más baja de memoria.





# Operadores de Memoria:

## ej.1

► Código C:

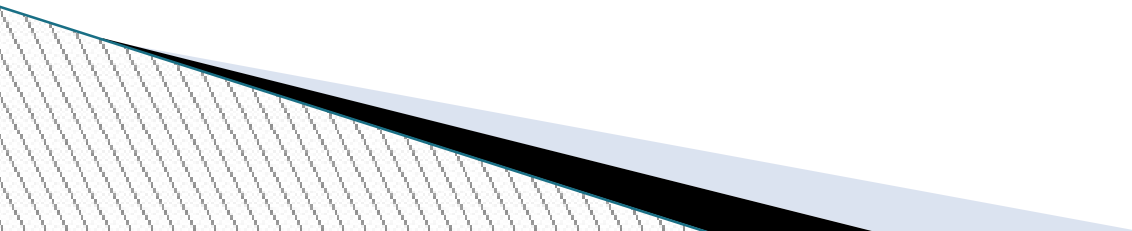
$g = h + A[8]$

$g$  en  $\$s1$     $h$  en  $\$s2$     $A(\text{inicio})$  en  $\$s3$

► Código MIPS:

`lw $t0, 32($s3)   # load`

`add $s1, $s2, $t0`



# Operadores de Memoria:

## ej.2

- ▶ Código C:

$A[10] = h + A[8]$

h en \$s2    A(inicio) en \$s3

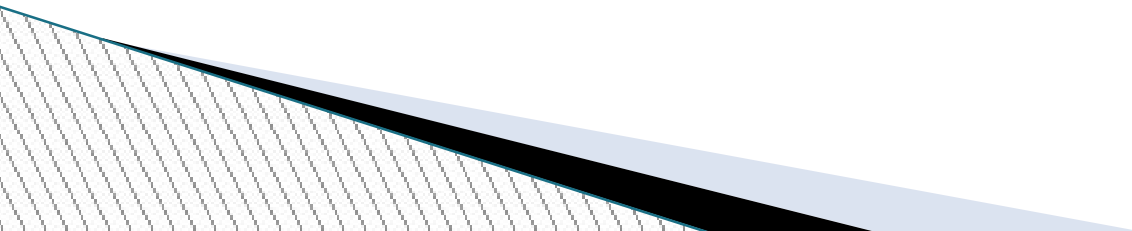
- ▶ Código MIPS:

lw \$t0, 32(\$s3)    # load

add \$t0, \$s2, \$t0

sw \$t0, 40(\$s3)    # store

# Memoria vs. Registro

- ▶ El acceso a registros es más rápido que a memoria.
  - ▶ Operar datos que se encuentran en memoria requiere de cargas y almacenamientos.
  - ▶ El compilador debe usar los registros para variables tanto como sea posible.
  - ▶ El compilador debe hacer un uso eficiente de los registros.
- 

# Operandos inmediatos

- Constantes especificadas en una instrucción:

Suma:

`addi $s2, $s3, 5`

Resta:

`addi $s3, $s3, -5`

- Principio de diseño 3: hacer rápido el caso común.

Operandos inmediatos evitan cargas de memorias.

# La constante cero

- El registro 0 de MIPS (\$zero) es la constante 0
- \$zero no puede ser reescrito

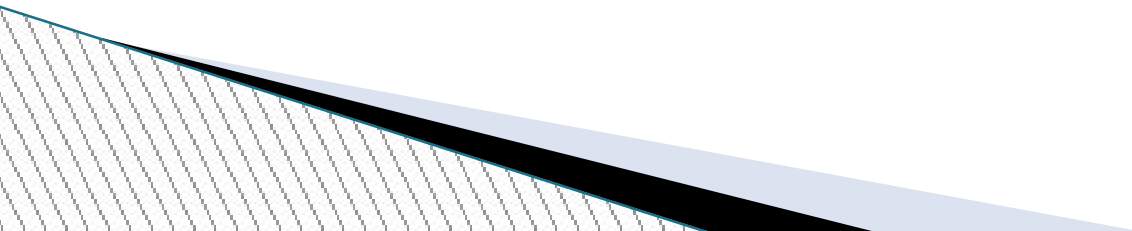
Para qué se usa \$zero?

Por ejemplo para mover el contenido de un registro a otro:

```
add $t0, $s3, $zero
```

# REPASO

- Hasta ahora que vimos?



# Estructuras de Control

- Instrucciones para la toma de decisiones:

**beq registro1, registro2, L1**

beq significa salta si es igual (branch if equal)

**bne registro1, registro2, L1**

bne significa salta sino es igual (branch if not equal)

Ambas instrucciones son llamadas saltos condicionales o bifurcaciones.

# Estructuras de Control: (cont.)

► Ejemplo:

Código en C:

```
if (i == j) f = g + h; else f = g - h;
```

Código MIPS: f, g, h, i, j en \$s0, \$s1, \$s2, \$s3, \$s4

```
bne $s3, $s4, Else
```

```
add $s0, $s1, $s2
```

```
j Exit          # ir a Exit
```

```
Else: sub $s0, $s1, $s2
```

```
Exit: .....
```

► Salto incondicional: j Exit



# Estructuras de Control (cont.)

- Bucles
- Estructura iterativa While: ejemplo

Código en C:

```
While (guardar[i] == k)  
    i += 1;
```

Código MIPS: suponiendo que i, k ocupan \$s3, \$s5 respectivamente y el inicio de guardar en \$s6 tenemos que:

Primero cargar a guardar[i] en un registro temporal.

# Estructuras de Control (cont.)

```
While: sll $t1, $s3, 2      # $t1 = i * 4
      add $t1, $t1, $s6     # $t1 = dir guardar[i]
      lw $t0, 0($t1)       # $t0 = guardar[i]
      bne $t0, $s5, FinW   # ir a FinW si guardar[i] != k
      addi $s3, $s3, 1     # i = i + 1
      j While              # ir a While
FinW: .....
```

# Estructuras de Control (cont.)

- Otro ejemplo: While (guardar[i] < k) i += 1;

```
While: sll $t1, $s3, 2      # $t1 = i * 4
      add $t1, $t1, $s6    # $t1 = dir guardar[i]
      lw $t0, 0($t1)      # $t0 = guardar[i]
      slt $s2, $t0, $s5   # $s2 = 1 si $t0 < $s5
      beq $s2, $zero, FinW # ir a FinW si $s2 = 0
      addi $s3, $s3, 1    # i = i + 1
      j While             # ir a While
```

FinW: .....

- ¿Cómo sería el siguiente bucle for?  
for (i=0; i<5; i++) h = h+i;

# Estructuras de Control

## (cont.)

- ¿Cómo sería el siguiente bucle for?(prop.Fernando Lomazzi)
- for (i=0; i<5; i++) h = h+i;  
i,h en \$S0,\$S1  
5 constante en \$T0

Código:

```
addi $S0,$zero,0   #inicializo S0 en 0
addi $T0,$zero,5   #inicializo T0 en 5
For: slt $S2,$S0,$T0 #S2=1 si S0<T0
      beq $S2,$zero,FinF #Salir si S2==1
      add $S1,$S1,$S0   #h=h+i
      addi $S0,$S0,1   #i+=1
```

j For

FinF:.....

# Soporte a procedimientos

## ► Pasos a seguir por el programa:

1. Colocar los parámetros en un lugar donde el procedimiento pueda acceder a ellos.
2. Transferir el control al procedimiento.
3. Ejecutar el código del procedimiento.
4. Colocar el resultado donde el programa tenga acceso a ellos.
5. Retornar el control al punto de origen.



1. \$a0..\$a3
2. jal direcciónProcedimiento  
(jal: jump and link. Instrucción de saltar y enlazar)
3. Ejecutar el código del procedimiento.
4. \$v0..\$v3
5. \$ra

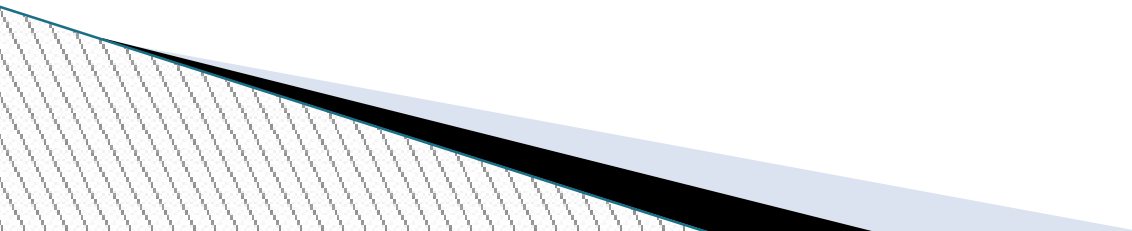
# Ejemplo de una función

- Código en C:

```
int ejemplo(int g, int h, int i, int j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

- Código MIPS:

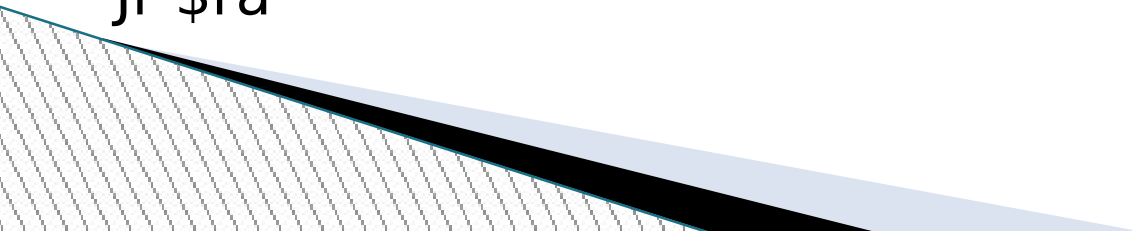
Las variables g, h, i, j se corresponden con \$a0, \$a1, \$a2, \$a3 respectivamente y f con \$s0.



# Ejemplo de una función (cont.)

ejemplo:

```
addi $sp, $sp, -12 #ajusta la pila para salvar 3 campos
sw $t1, 8($sp)
sw $t0, 4($sp)
sw $s0, 0($sp)
add $t0, $a0, $a1
add $t1, $a2, $a3
sub $s0, $t0, $t1
add $v0, $s0, $zero
lw $s0, 0($sp)
lw $t0, 4($sp)
lw $t1, 8($sp)
jr $ra
```



# Convenciones MIPS

- ▶ En relación a los 18 registros, MIPS los separa en dos grupos:
- ▶ `$t0..$t9`: no preservado por el procedimiento llamado.
- ▶ `$s0..$s7`: se deben preservar en una llamada a procedimiento en caso de ser usados.

Entonces teniendo en cuenta esta convención el ejemplo anterior queda:

```
addi $sp, $sp, -4 #ajusta la pila para salvar 1 campo
sw $s0, 0($sp)
add $t0, $a0, $a1
add $t1, $a2, $a3
sub $s0, $t0, $t1
add $v0, $s0, $zero
lw $s0, 0($sp)
jr $ra
```

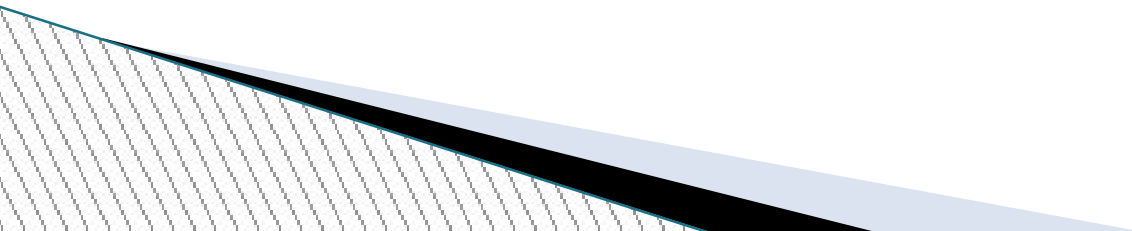


# Procedimientos Anidados

- Para las llamadas anidadas, el procedimiento que llama tiene que guardar en la pila:
- Su dirección de retorno.
- Cualquier argumento o temporales que va a necesitar después de la llamada.
- Restaura desde la pila después de la llamada.

Ejemplo: código en C

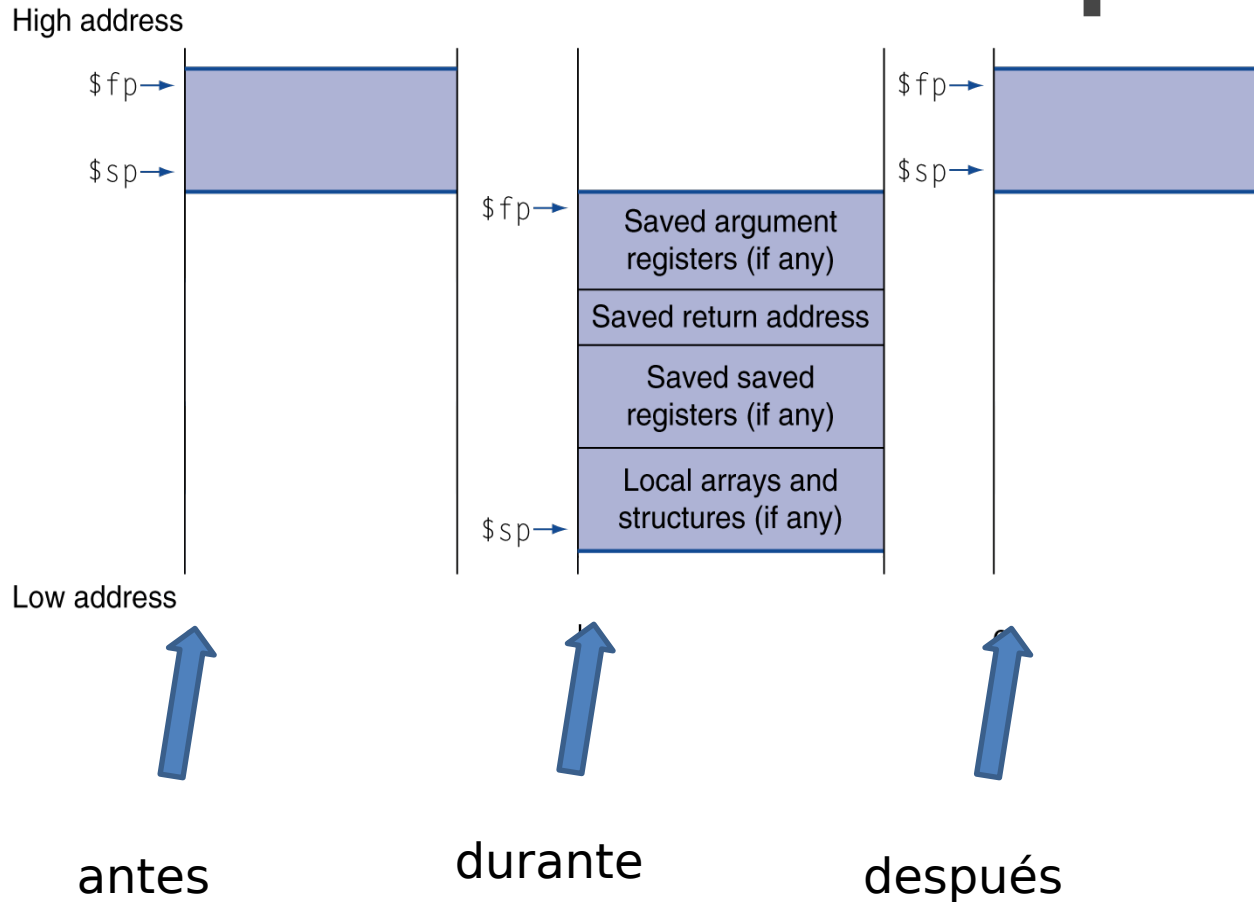
```
int fact (int n)
{ if (n<1) return 1;
  else return (n* fact(n-1));
}
```



# Procedimientos Anidados (ejem.)

- ▶ Código MIPS: ponemos el argumento  $n$  en  $\$a0$  y el resultado en  $\$v0$
- ▶ fact:
  - ▶ `addi $sp, $sp, -8`    # ajusta la pila para guardar 2 items
  - ▶ `sw $ra, 4($sp)`    # guarda dirección de retorno
  - ▶ `sw $a0, 0($sp)`    # guarda el argumento  $n$
  - ▶ `slti $t0, $a0, 1`    # test para  $n < 1$
  - ▶ `beq $t0, $zero, L1` # si  $n \geq 1$ , ir a L1
  - ▶ `addi $v0, $zero, 1` # resultado 1
  - ▶ `addi $sp, $sp, 8`    # elimina 2 items de la pila
  - ▶ `jr $ra`    # retorna al llamador
- ▶ L1: `addi $a0, $a0, -1` # decrementa  $n$
- ▶ `jal fact`    # llamar a fact con  $(n-1)$
- ▶ `lw $a0, 0($sp)`    # restaura el argumento  $n$
- ▶ `lw $ra, 4($sp)`    # restaura la dirección de retorno
- ▶ `addi $sp, $sp, 8`    # elimina 2 items de la pila
- ▶ `mul $v0, $a0, $v0` # multiplica  $n * \text{fact}(n-1)$
- ▶ `jr $ra`    # retorna al llamador

# Asignación del espacio para los nuevos datos en la pila:



# Convenciones de registro MIPS

| NOMBRE    | NUMERO | USO                                                      | PRESERVADO EN LLAMADA |
|-----------|--------|----------------------------------------------------------|-----------------------|
| \$zero    | 0      | Valor constante cero                                     | n.a.                  |
| \$v0-\$v1 | 2-3    | Valores para los resultados y evaluación de la expresión | no                    |
| \$a0-\$a3 | 4-7    | Argumentos                                               | no                    |
| \$t0-\$t7 | 8-15   | Temporales                                               | no                    |
| \$s0-\$s7 | 16-23  | Guardados                                                | sí                    |
| \$t8-\$t9 | 24-25  | Más temporales                                           | no                    |
| \$gp      | 28     | Puntero global                                           | sí                    |
| \$sp      | 29     | Puntero de la pila                                       | sí                    |
| \$fp      | 30     | Puntero de estructura                                    | sí                    |
| \$ra      | 31     | Dirección de retorno                                     | sí                    |

# Resumen: Operandos MIPS

| Nombre                        | ejemplo                                                                                       | comentarios                                                                                                                                                                                                                                                                              |
|-------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 registros                  | <code>\$s0-\$s7, \$t0-\$t7, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code> | Localizaciones rápidas para los datos. En MIPS los datos deben estar en los registros para realizar operaciones aritméticas. El registro MIPS <code>\$zero</code> es siempre igual a 0. El registro <code>\$at</code> esta reservado por el ensamblador para manejar grandes constantes. |
| $2^{30}$ palabras de memorias | <code>Memory[0], Memory[4], . . . , Memory[4294967292]</code>                                 | Accesibles solamente por instrucciones de transferencia de datos. MIPS utiliza direcciones de byte, de modo que las direcciones de palabras consecutivas difieren en 4. La memoria guarda las estructuras de datos, las tablas y los registros desbordados (guardados).                  |

# Resumen: Lenguaje ensamblador MIPS

## ► Categoría: **Aritmética**

| Instrucción   | ejemplo              | significado          | comentarios                      |
|---------------|----------------------|----------------------|----------------------------------|
| add           | add \$s1, \$s2, \$s3 | $\$s1 = \$s2 + \$s3$ | 3 operandos. datos en registros. |
| subtract      | sub \$s1, \$s2, \$s3 | $\$s1 = \$s2 - \$s3$ | 3 operandos. datos en registros. |
| add immediate | addi \$s1, \$s2, 100 | $\$s1 = \$s2 + 100$  | 3 operandos. datos en registros. |

# Resumen: Lenguaje ensamblador MIPS(cont.)

## ► Categoría: **Transferencia de datos.**

| Instrucción      | ejemplo            | significado                        | comentarios                                    |
|------------------|--------------------|------------------------------------|------------------------------------------------|
| load word        | lw \$s1, 100(\$s2) | $\$s1 = \text{Memory}[\$s2 + 100]$ | Palabra de memoria a registro.                 |
| store word       | sw \$s1, 100(\$s2) | $\text{Memory}[\$s2 + 100] = \$s1$ | Palabra de registro a memoria.                 |
| load half        | lh \$s1, 100(\$s2) | $\$s1 = \text{Memory}[\$s2 + 100]$ | Media palabra de memoria a registro.           |
| store half       | sh \$s1, 100(\$s2) | $\text{Memory}[\$s2 + 100] = \$s1$ | Media palabra de registro a memoria.           |
| load byte        | lb \$s1, 100(\$s2) | $\$s1 = \text{Memory}[\$s2 + 100]$ | Byte de memoria a registro.                    |
| store byte       | sb \$s1, 100(\$s2) | $\text{Memory}[\$s2 + 100] = \$s1$ | Byte de registro a memoria.                    |
| load upper immed | lui \$s1, 100      | $\$s1 = 100 * 2^{16}$              | Cargar constante en los 16 bits de mayor peso. |

# Resumen: Lenguaje ensamblador MIPS(cont.)

## ► Categoría: **Lógica.**

| Instrucción         | ejemplo              | significado                   | comentarios                               |
|---------------------|----------------------|-------------------------------|-------------------------------------------|
| and                 | and \$s1, \$s2, \$s3 | $\$s1 = \$s2 \& \$s3$         | 3 operandos; and bit a bit.               |
| or                  | or \$s1, \$s2, \$s3  | $\$s1 = \$s2 \mid \$s3$       | 3 operandos; or bit a bit.                |
| nor                 | nor \$s1, \$s2, \$s3 | $\$s1 = \sim(\$s2 \mid \$s3)$ | Tres operandos; nor bit a bit.            |
| and immediate       | andi \$s1, \$s2, 100 | $\$s1 = \$s2 \& 100$          | And bit a bit registro con constante.     |
| or immediate        | ori \$s1, \$s2, 100  | $\$s1 = \$s2 \mid 100$        | Or bit a bit registro con constante.      |
| shift left logical  | sll \$s1, \$s2, 10   | $\$s1 = \$s2 \ll 10$          | Desplazamiento a izquierda por constante. |
| shift right logical | srl \$s1, \$s2, 10   | $\$s1 = \$s2 \gg 10$          | Desplazamiento a derecha por constante.   |



# Resumen: Lenguaje ensamblador MIPS(cont.)

## ► Categoría: **Salto condicional.**

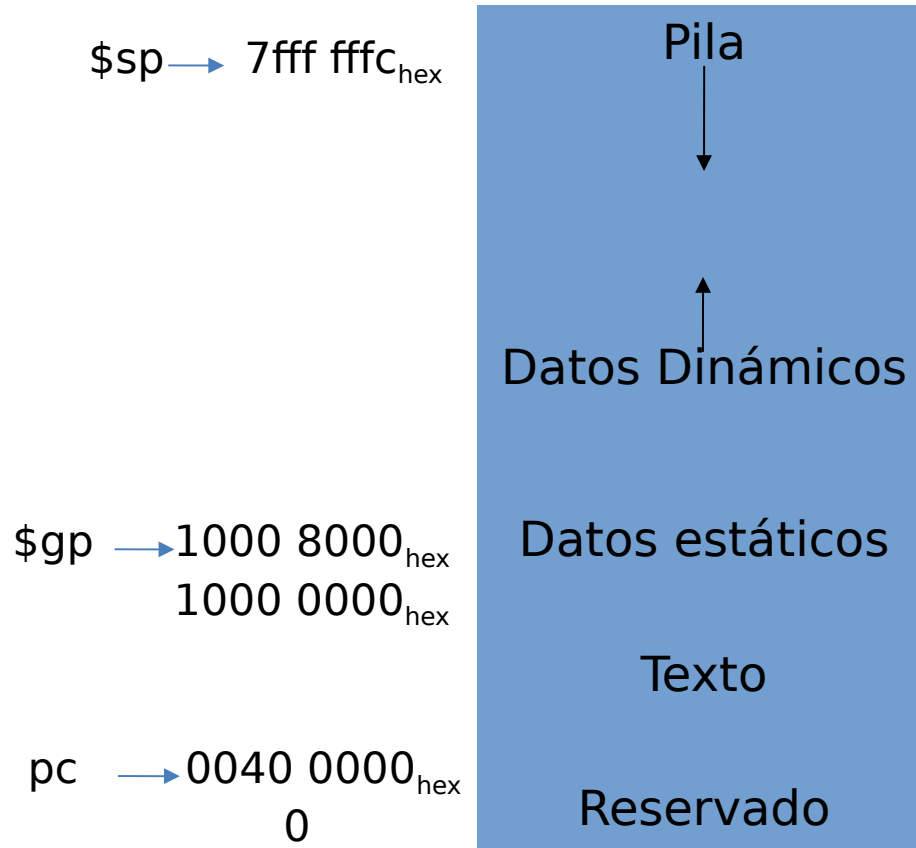
| Instrucción                | ejemplo              | significado                                 | comentarios                                        |
|----------------------------|----------------------|---------------------------------------------|----------------------------------------------------|
| branch on equal            | beq \$s1, \$s2, L    | if (\$s1==\$s2) go to L<br>PC + 4 + 100     | Comprueba igualdad y bifurca relativo al PC.       |
| branch on not equal        | bne \$s1, \$s2,L     | if (\$s1!=\$s2) go to L<br>PC + 4 + 100     | Comprueba si no es igual y bifurca relativo al PC. |
| set on less than           | slt \$s1, \$s2, \$s3 | if (\$s2 < \$s3) \$s1 = 1;<br>else \$s1 = 0 | Compara si es menor a que; usado beq, bne.         |
| set on less than immediate | slti \$s1, \$s2, 100 | if (\$s2 < 100) \$s1 = 1;<br>else \$s1 = 0  | Compara si es menor que una constante.             |

# Resumen: Lenguaje ensamblador MIPS(cont.)

► Categoría: **Salto incondicional.**

| Instrucción   | ejemplo  | significado                | comentarios                    |
|---------------|----------|----------------------------|--------------------------------|
| jump          | j 2500   | go to 10000                | Salto a la dirección destino.  |
| jump register | jr \$ra  | go to \$ra                 | Para retorno de procedimiento. |
| jump and link | jal 2500 | \$ra = PC + 4; go to 10000 | Para llamada a procedimiento.  |

# Convención MIPS de asignación de memoria



# FIN

## 1ERA. PARTE

